Arxiv Research Paper Classifier System Design Document

Jayaharish R EP21B016

April 28, 2026

Contents

1 Introduction **3**

1.1 Purpose 3

1.2 Scope 3

2 High-Level Design (HLD) **3**

2.1 System Overview 3

2.2 Architecture 3

2.3 Core Components 4

2.3.1 Frontend (Streamlit) 4

2.3.2 Backend API 4

2.3.3 Model Serving Layer (MLflow) 4

2.3.4 MLflow Tracking 5

2.3.5 Monitoring and Observability Layer 5

2.4 Data Flow 5

2.5 End-to-End Flow 6

2.6 Monitoring and Observability 6

2.6.1 Metrics Collection 6

2.6.2 Visualization 6

2.6.3 Alerting and Notification 6

2.6.4 Application-Level Observability 7

2.6.5 Operational Impact 7

3 Low-Level Design (LLD) 7

3.1 Backend API Design 7

3.1.1 Health Check Endpoint 7

3.1.2 Single Prediction Endpoint 8

3.1.3 Bulk Prediction Endpoint 9

3.2 Model Serving Architecture 10

3.3 Frontend Interaction Flow 10

3.3.1 Single Prediction Flow 11

3.3.2 Bulk Prediction Flow 11

3.4 Execution Characteristics 11

3.5 Failure Handling Strategy 12

4 Deployment and Communication 12

4.1 System Setup 12

4.2 Communication Flow 12

4.3 Request Lifecycle 12

5 Design Decisions 12

6 Conclusion 13

1 Introduction

1.1 Purpose

This document presents the detailed system design of the Arxiv Research Paper Classifier. It focuses on architectural decisions, system components, interfaces, and execution flow.

1.2 Scope

This document includes:

- High-Level Architecture
- Component Interaction
- API Design (LLD)
- Deployment and communication flow

2 High-Level Design (HLD)

2.1 System Overview

The system is designed to classify research papers into relevant categories using their title and abstract. It supports both single input predictions and batch processing through a CSV interface.

The architecture follows a modular and service-oriented design, where the frontend, backend, model serving layer, and monitoring stack operate independently while communicating through well-defined interfaces.

2.2 Architecture

The system processes requests through a structured pipeline consisting of input handling, validation, preprocessing, model inference, and response generation.

In parallel, a monitoring and observability stack continuously tracks system performance and health.

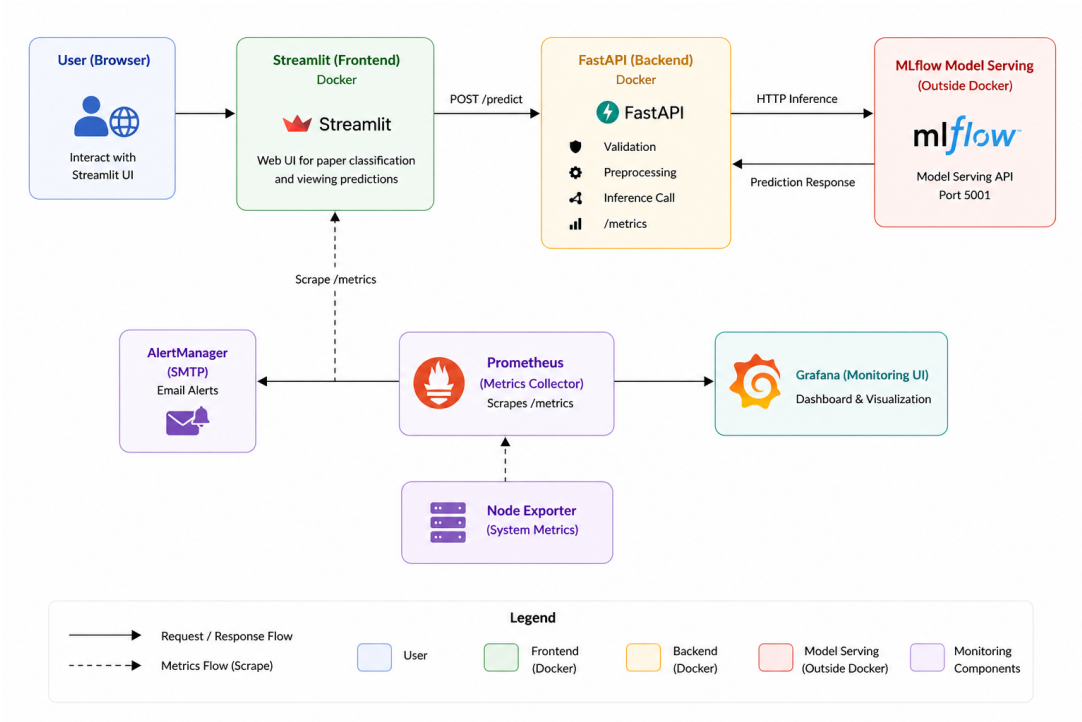

Figure 1: High-Level Architecture

2.3 Core Components

2.3.1 Frontend (Streamlit)

The frontend provides the user interface for interacting with the system.

It supports both single prediction and bulk prediction workflows.

The interface also communicates system states such as initialization, processing, and result display, ensuring that users are aware of ongoing operations.

2.3.2 Backend API

The backend acts as the central orchestration layer.

It handles incoming requests, validates user input, performs preprocessing, and communicates with the model serving layer.

It is responsible for maintaining a consistent execution flow and handling errors gracefully.

2.3.3 Model Serving Layer (MLflow)

The trained model is deployed using `mlflow models serve`, which exposes a REST endpoint for inference.

The backend sends processed inputs to this endpoint and receives predictions.

This decouples model inference from application logic and allows the model to be loaded once and reused across requests.

2.3.4 MLflow Tracking

MLflow is used to track experiments, parameters, and evaluation metrics during training.

It stores model artifacts and enables reproducibility and comparison across different runs.

2.3.5 Monitoring and Observability Layer

The monitoring layer provides visibility into system performance, resource usage, and operational health.

It integrates metrics collection, visualization, and alerting tools to ensure that the system remains observable and reliable.

2.4 Data Flow

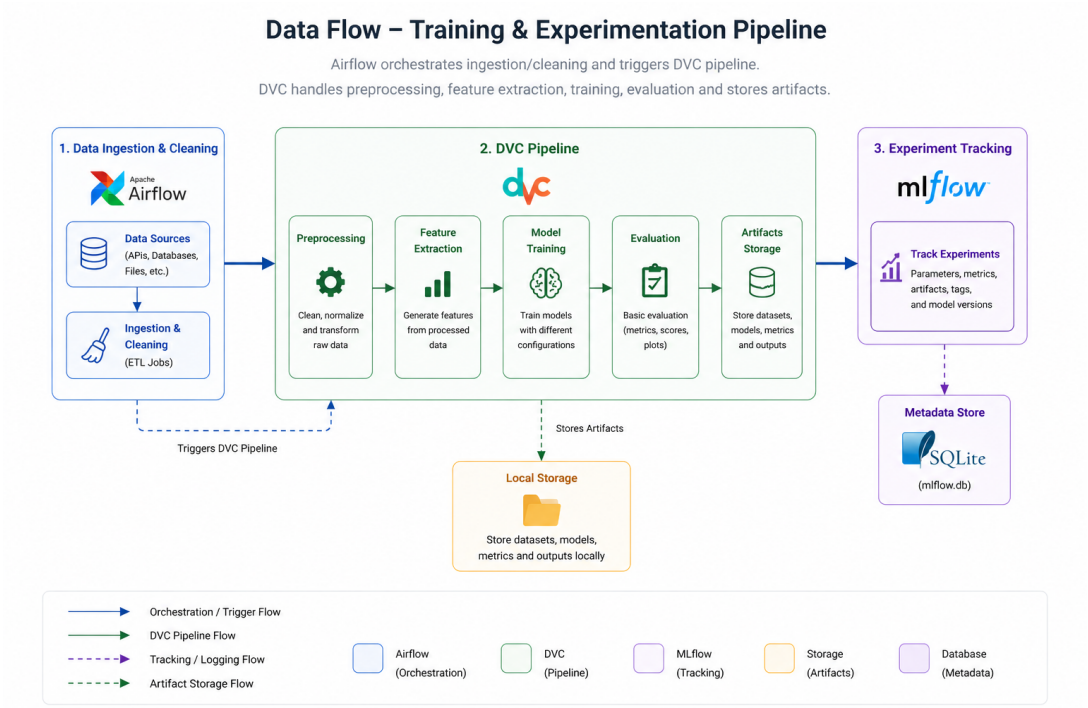

Figure 2: System Data Flow

The system follows a linear data flow from user input to prediction output.

Monitoring components operate alongside this flow to continuously observe system behavior.

2.5 End-to-End Flow

1. User provides input (title and abstract or CSV file)
2. Frontend sends a request to the backend API
3. Backend validates and preprocesses input data
4. Processed input is sent to the MLflow serving endpoint
5. Model performs inference and returns predictions
6. Backend formats the output
7. Results are displayed on the frontend

2.6 Monitoring and Observability

Monitoring is implemented as an integrated layer that operates alongside the core system to provide real-time visibility into performance and health.

2.6.1 Metrics Collection

Prometheus is used as the primary metrics collection system.

It periodically scrapes metrics from the backend service, capturing information such as request counts, inference latency, and error rates.

Node Exporter is used to collect system-level metrics, including CPU usage, memory utilization, and disk activity.

These metrics help identify resource bottlenecks during execution.

2.6.2 Visualization

Grafana is used to visualize the collected metrics.

Dashboards provide insights into system health, request throughput, latency trends, and resource utilization.

This enables real-time monitoring and historical analysis of system performance.

2.6.3 Alerting and Notification

Alertmanager is used to define and manage alert rules based on system metrics.

Alerts are triggered under conditions such as high latency, increased error rates, or excessive resource usage.

Notifications are routed through an email system using Mailtrap, which allows safe testing of alert delivery without sending real emails.

2.6.4 Application-Level Observability

In addition to infrastructure monitoring, the system provides application-level feedback.

The frontend displays user-facing status indicators such as:

- "Model is initializing"
- "Analyzing..."
- "Processing batch..."

The backend logs request handling, validation failures, and prediction execution, enabling debugging and traceability.

2.6.5 Operational Impact

The monitoring setup ensures that system behavior is continuously observable.

It enables early detection of failures, identification of performance bottlenecks, and automated alerting for critical issues.

This improves system reliability, maintainability, and ease of debugging.

3 Low-Level Design (LLD)

3.1 Backend API Design

The backend exposes a set of RESTful endpoints that handle health checks, single predictions, and batch predictions. These endpoints are designed to be stateless and operate independently for each request.

3.1.1 Health Check Endpoint

Endpoint: `GET /health`

Description:

This endpoint is used by the frontend to verify whether the backend service and model are fully initialized and ready to handle requests.

It is called periodically during application startup to ensure that predictions are only enabled once the system is operational.

Response:

```
{
  "status": "ready"
}
```

Failure Behavior:

- If the model is not yet loaded, the endpoint may return a non-ready state.
- If the backend is unreachable, the frontend handles this as a connectivity failure.

3.1.2 Single Prediction Endpoint

Endpoint: `POST /predict`

Description:

This endpoint processes a single research paper by accepting its title and abstract and returning predicted categories.

It is the primary endpoint used in interactive mode via the frontend.

Request:

```
{
  "title": "string",
  "abstract": "string"
}
```

Validation:

- Title must be non-empty and exceed a minimum length threshold
- Abstract must be sufficiently descriptive
- Invalid inputs are rejected before model inference

Processing Pipeline:

- Input validation
- Text preprocessing (cleaning, normalization)
- Feature extraction
- Model inference
- Post-processing of predictions

Response:

```
{
  "labels": ["cs.LG", "stat.ML"]
}
```

Error Handling:

- Invalid input returns a descriptive error message
- Internal failures return a generic prediction failure response

3.1.3 Bulk Prediction Endpoint

Endpoint: `POST /predict-batch`

Description:

This endpoint enables batch processing of multiple research papers using a CSV file.

It is designed for higher throughput scenarios where multiple predictions are required in a single request.

Input Requirements:

- File format: CSV
- Required columns:
  - `title`
  - `abstract`

Processing Pipeline:

- File validation and parsing
- Iteration over each row
- Row-wise preprocessing
- Batch inference
- Aggregation of results

Response:

- A structured output containing predictions for each row

Error Handling:

- Invalid file format or missing columns results in rejection
- Individual row failures are handled without stopping the entire batch (where possible)

Performance Considerations:

- Processing time increases with file size
- Large batches may introduce higher latency

3.2 Model Serving Architecture

The trained model is served using MLflow's model serving capability.

- The model is deployed using `mlflow models serve`
- This exposes a REST API endpoint for inference
- The backend communicates with this service instead of directly loading the model into memory

Serving Characteristics:

- The model is loaded once at startup and reused for all requests
- Inference is performed via HTTP calls to the MLflow serving endpoint
- This separation allows independent scaling of the model serving layer

Example Inference Flow:

1. Backend receives request from frontend
2. Backend preprocesses input data
3. Processed input is sent to MLflow serving endpoint
4. MLflow model returns predictions
5. Backend formats and returns response to frontend

3.3 Frontend Interaction Flow

The frontend interacts with the backend through synchronous HTTP requests and updates the UI based on the response.

3.3.1 Single Prediction Flow

1. User selects Single Prediction mode
2. User enters title and abstract
3. Frontend performs basic validation (e.g., non-empty fields)
4. Request is sent to `/predict`
5. Backend processes the input and returns predicted labels
6. Frontend displays results to the user

User Feedback:

- "Analyzing..." is displayed during processing
- Errors are shown if validation or prediction fails

3.3.2 Bulk Prediction Flow

1. User selects Bulk Prediction mode
2. User uploads CSV file
3. Frontend validates file structure
4. File is sent to `/predict-batch`
5. Backend processes all rows and generates predictions
6. Results are returned and displayed in tabular format

User Feedback:

- "Processing batch..." is displayed during execution
- Errors related to file format or processing are surfaced clearly

3.4 Execution Characteristics

- All endpoints are stateless and independent
- Each request is processed synchronously
- Model inference is performed in real-time
- No persistent session state is maintained between requests

3.5 Failure Handling Strategy

- Input validation prevents invalid requests from reaching the model
- Backend exceptions are logged and handled gracefully
- Frontend provides clear feedback to users in case of failure

4 Deployment and Communication

4.1 System Setup

The system is deployed using Docker containers.

- Frontend: Streamlit application
- Backend: API service
- MLflow: Experiment tracking (local)

4.2 Communication Flow

- Frontend communicates with backend via HTTP
- Backend performs inference locally
- MLflow is used for offline tracking

4.3 Request Lifecycle

1. User submits input
2. Frontend sends API request
3. Backend processes and predicts
4. Response returned to frontend

5 Design Decisions

- Separation of frontend and backend for modularity
- REST API for communication
- Batch processing support for scalability
- MLflow used for experiment tracking and reproducibility

6 Conclusion

The system is designed to provide a clean, modular, and scalable solution for research paper classification. The architecture ensures separation of concerns, ease of deployment, and reproducibility through experiment tracking.